

CS 131 Problem Set 1: *Results and Discussion*

Joshua Frankie B. Rayo, Edward Nataniel C. Apostol
CS 131 WFY

February 26, 2013

1 SLE vs SNLE

1.1 Matrix Inverse

The inverse of a nonsingular matrix needs to be explicitly computed because of several reasons. We really need to find its inverse and they are required in order for us to move forward. When we are calculating the inverse of a matrix, we are also studying its properties like the determinant, rank and adjoint. We also use the inverse of a matrix for solving systems of equations, linear or nonlinear. One important application of matrix inverse is cryptography [<http://aix1.uottawa.ca/jkhoury/cryptography.htm>]. Given a message matrix M , encryption matrix A is used for encoding so that the encrypted message is AM . For decryption, A^{-1} is used so that $A^{-1}(AB) = B$.

1.2 Matrix Inverse as System of Linear Equations

Finding the matrix inverse is also equivalent to simultaneous solution of systems of linear equations. We want to satisfy the equation $AX = I$, where I is an identity matrix. Finding X is equivalent to finding the inverse of A . Since I is a square matrix, the system $AX = I$ is actually a simultaneous system of linear equations. We let $I = [i_1 \ i_2 \ \dots \ i_n]$. Actually we are solving for x_k 's in the system $Ax_1 = i_1$, $Ax_2 = i_2$, ..., $Ax_n = i_n$. When we obtained all the x_k 's, $X = [x_1 \ x_2 \ \dots \ x_n]$ and X is actually equal to A^{-1} .

1.3 Matrix Inverse as System of Nonlinear Equations

Chapter 5 Computer Problem 5.17 of Heath (First Ed) shows another way on how we compute the inverse of a matrix using nonlinear systems of equations. Consider the equation $AX = I$. We want to find the inverse of A . It can be rewritten as $0 = I - AX$ and we let a function

$$\mathbf{F}(\mathbf{X}) = \mathbf{I} - \mathbf{A}\mathbf{X}$$

We know that $\mathbf{F}(\mathbf{X}) = \mathbf{0}$ if $\mathbf{X} = \mathbf{A}^{-1}$. Finding its root is equivalent to finding the inverse of $\mathbf{A}$. So we can use root-finding methods particularly Newton's method such that $\mathbf{F}(\mathbf{X}) = \mathbf{0}$. We recall the Newton's iterative scheme for systems of nonlinear equations:

$$\mathbf{X}_{k+1} = \mathbf{X}_k - [\mathbf{F}'(\mathbf{X}_k)]^{-1} \mathbf{F}(\mathbf{X}_k)$$

For our specific problem, the Newton's iterative scheme is

$$\mathbf{X}_{k+1} = \mathbf{X}_k + \mathbf{A}^{-1}(\mathbf{I} - \mathbf{A}\mathbf{X}_k)$$

But $\mathbf{A}^{-1}$ is what we are solving, so we replace it by our approximation $\mathbf{X}_k$:

$$\mathbf{X}_{k+1} = \mathbf{X}_k + \mathbf{X}_k(\mathbf{I} - \mathbf{A}\mathbf{X}_k)$$

And the final form of Newton's iteration scheme for this problem is

$$\mathbf{X}_{k+1} = \mathbf{X}_k(2\mathbf{I} - \mathbf{A}\mathbf{X}_k) \tag{1}$$

Analogously, if we let our function $\mathbf{F}(\mathbf{X}) = \mathbf{A}\mathbf{X} - \mathbf{I} = \mathbf{0}$, our iterative scheme is also the same. Now we must prove the convergence of this iterative method. Heath defines the residual matrix as

$$\mathbf{R}_k = \mathbf{I} - \mathbf{A}\mathbf{X}_k \tag{2}$$

and the error matrix as

$$\mathbf{E}_k = \mathbf{A}^{-1} - \mathbf{X}_k. \tag{3}$$

Recall that an iterative method is said to have convergence of order $q \geq 1$ if for asymptotic error constant $0 < c < \infty$,

$$\epsilon_{n+1} = c\epsilon_n^q$$

when n is large. We evaluate $\mathbf{R}_{k+1}$ using equation (1) and equation (2):

$$\begin{aligned} R_{k+1} &= I - AX_{k+1} \\ &= I - A[X_k(2I - AX_k)] \\ &= I - A(2X_k - AX_k^2) \\ &= I - 2AX_k + [AX_k]^2. \end{aligned}$$

We recall a property of matrices:

$$(A + B)^2 = A^2 + AB + BA + B^2.$$

We evaluate $\mathbf{R}_k^2$ using equation (2) and apply the property given above:

$$\begin{aligned} R_k^2 &= (I - AX_k)(I - AX_k) \\ &= I^2 - IAX_k - AX_kI - [AX_k]^2 \\ &= I - 2AX_k + [AX_k]^2. \end{aligned}$$

We evaluate $\mathbf{E}_{k+1}$ using equation (1) and equation (3):

$$\begin{aligned} E_{k+1} &= A^{-1} - X_{k+1} \\ &= A^{-1} - [X_k(2I - AX_k)] \\ &= A^{-1} - (2IX_k - AX_k^2) \\ &= A^{-1} - 2X_k + AX_k^2. \end{aligned}$$

Lastly, we evaluate $\mathbf{E}_k\mathbf{A}\mathbf{E}_k$ using equation (3):

$$\begin{aligned} E_kAE_k &= (A^{-1} - X_k)A(A^{-1} - X_k) \\ &= (A^{-1} - X_k)(I - AX_k) \\ &= A^{-1} - X_k - X_k + AX_k^2 \\ &= A^{-1} - 2X_k + AX_k^2. \end{aligned}$$

Given these relationships, we can conclude that

$$\mathbf{R}_{k+1} = \mathbf{R}_k^2 \text{ and } \mathbf{E}_{k+1} = \mathbf{E}_k\mathbf{A}\mathbf{E}_k = \mathbf{A}\mathbf{E}_k^2$$

and the iterative method for matrix inversion has quadratic convergence, with the original matrix $\mathbf{A}$ as the asymptotic error constant.

1.4 Comparison of Matrix Inversion Methods

Known methods for finding the inverse of a matrix are the Gauss-Jordan Reduction method, LU Factorization method and the Iterative Scheme discussed above. The attached file *ProblemOne.sce* contains the source code that implements the methods mentioned above. Our program generates a random integer $n \in [5, 20]$ and generates a random $n \times n$ matrix with random integer entries on $[-10, 10]$. We will compare the execution time of these methods for computing its inverse and gather the results. For us to have a good comparison of the methods, we assume that the matrix to be used is *nice*. We define a *nice* matrix as follows:

- The matrix is not singular.
- The LU decomposition of the matrix produces a permutation matrix equal to the identity matrix. This means that we do not have to rearrange the rows of a matrix to decompose it into lower and upper matrices.

If a randomly generated matrix does not belong to *nice* matrices, we generate a matrix again for us to continue. We used the `grand()` function of Scilab to generate random matrices.

1.4.1 Gauss-Jordan Reduction Method

We know that numbers like 2.220D-16 and -4.441D-16 are produced by Scilab because of its poor precision. To handle this situation, we modified the condition for GJR method:

```
if abs(M(k, k)) <= 1D-10 then error('singular matrix!'); end
```

If a pivot has value practically zero then the matrix is singular and we abort the matrix inversion.

1.4.2 LU Factorization Method

Since this is not discussed in class, we provide the algorithm here.

1. Given a non-singular matrix $\mathbf{A}$, we decompose it as $\mathbf{A} = \mathbf{LU}$ into lower and upper triangular matrices.
2. We solve the system $\mathbf{Lb}_1 = \mathbf{I}_1$, $\mathbf{Lb}_2 = \mathbf{I}_2$, ..., $\mathbf{Lb}_n = \mathbf{I}_n$ using forward substitution. $\mathbf{I}_i$'s are the corresponding column vectors of the identity matrix $\mathbf{I}$.
3. Then we solve the system $\mathbf{Ux}_1 = \mathbf{b}_1$, $\mathbf{Ux}_2 = \mathbf{b}_2$, ..., $\mathbf{Ux}_n = \mathbf{b}_n$ using back substitution.
4. Matrix $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \dots \ \mathbf{x}_n]$ is now the inverse of $\mathbf{A}$.

1.4.3 Newton's Iterative Method

Heath cited that a reasonable starting guess for approximating the inverse is to take

$$\mathbf{X}_0 = \frac{\mathbf{A}^T}{\|\mathbf{A}\|_1 \cdot \|\mathbf{A}\|_\infty}.$$

So we take it as our starting guess and we implemented the iteration scheme discussed earlier. The iteration will stop if $\|\mathbf{X}_{k+1} - \mathbf{X}_k\|_1 < 1 \times 10^{-3}$. Our program will display the number of iterations before the stopping condition is satisfied.

1.5 Results and Observations

We used the `tic()` and `toc()` functions of Scilab to obtain the execution time of each method discussed to be used for comparison. The machine used in running the script uses Intel® Core™ i3-2310 CPU @ 2.10GHz with 8.00GB of RAM. We gather here five random *nice* matrices and time elapsed for each methods.

Matrix 1:

$$\begin{bmatrix} -10 & 0 & -1 & 8 & -10 & 0 & 6 & -8 & 2 & 0 & 8 & -2 & 7 & 7 & 5 & -10 & 6 & 9 & -5 & 4 \\ -2 & -8 & -8 & -9 & -6 & 6 & 1 & 9 & -1 & -10 & -8 & -10 & -5 & -7 & -1 & 7 & 2 & 1 & -10 & -10 \\ -2 & 6 & 8 & 9 & -3 & -7 & 8 & -1 & 4 & -6 & 2 & 10 & 3 & 7 & -2 & 10 & 8 & -2 & 2 & -6 \\ 6 & -4 & 10 & -10 & -2 & -7 & -1 & 7 & -6 & -8 & 3 & -4 & -4 & 7 & -3 & -9 & 9 & 3 & 5 & -5 \\ 3 & 4 & 5 & -7 & 5 & -7 & 2 & 5 & -5 & 6 & 5 & 6 & -6 & 10 & -6 & 7 & -3 & 5 & -4 & 5 \\ -8 & 6 & -7 & 6 & -3 & 4 & 0 & -6 & -5 & 10 & -4 & -6 & 0 & 7 & 1 & 9 & 10 & -1 & 2 & 1 \\ -5 & -4 & -3 & -4 & 7 & -7 & -10 & 4 & 4 & 6 & -9 & -3 & 0 & -10 & -10 & -6 & 2 & 4 & -9 & -1 \\ -10 & 6 & -6 & 2 & 5 & -2 & -8 & 6 & 10 & 1 & 1 & -5 & -10 & 10 & 1 & -7 & 8 & -6 & -6 & 4 \\ -6 & 10 & -8 & -4 & 2 & -3 & 3 & -7 & -7 & 4 & 5 & -2 & 9 & -9 & -3 & 8 & 7 & 8 & 7 & 6 \\ 3 & 8 & -9 & 7 & -8 & 9 & 10 & 8 & 4 & 3 & -8 & 1 & 10 & 9 & 0 & -1 & -1 & -10 & 0 & 0 \\ 7 & -10 & 1 & 6 & 1 & -6 & -8 & 3 & 4 & 10 & -10 & -5 & -7 & -6 & -4 & -5 & -4 & -4 & 3 & -3 \\ 8 & -9 & 4 & 5 & -9 & -8 & -3 & -7 & -2 & 7 & 10 & 6 & -6 & 1 & -1 & 6 & -8 & -2 & -4 & -8 \\ -10 & -8 & 2 & 9 & -10 & 2 & -7 & 5 & 9 & 3 & -8 & -9 & -3 & -4 & 8 & -10 & 2 & 5 & -2 & 10 \\ 10 & 5 & -1 & 2 & 5 & -9 & 6 & -7 & -5 & 2 & 5 & 0 & -10 & -5 & -8 & -5 & -8 & 9 & -1 & 8 \\ -10 & 3 & -7 & 4 & -3 & -7 & -2 & -10 & 7 & 6 & 6 & 9 & 0 & 6 & 1 & -5 & -8 & 5 & -3 & -9 \\ -4 & -3 & -9 & 7 & 0 & 5 & -2 & 4 & -4 & -7 & -1 & -6 & 7 & -1 & 0 & -1 & -9 & -10 & 2 & 8 \\ 0 & 10 & 1 & -5 & -5 & -2 & 4 & 8 & 3 & 9 & 9 & -6 & -1 & -9 & 4 & 0 & 4 & 1 & 2 & 7 \\ -9 & 6 & -3 & 9 & -9 & 6 & -9 & -7 & 9 & 9 & -3 & 10 & 4 & 7 & 5 & 5 & -9 & -1 & 0 & -1 \\ 9 & -7 & 4 & 3 & -10 & -7 & 2 & 7 & -1 & -7 & -2 & -10 & 4 & 1 & -2 & 1 & 10 & -7 & -10 & 3 \\ -2 & -10 & -6 & 4 & 1 & -7 & -1 & -2 & 9 & 8 & 3 & -2 & -5 & 10 & -4 & -10 & 10 & -7 & 10 & 5 \end{bmatrix}$$

Matrix 2:

$$\begin{bmatrix} 8 & -1 & 4 & -9 & -8 & -7 & 1 & -2 \\ 2 & 0 & -4 & -9 & -4 & 9 & -6 & -1 \\ -9 & 5 & -8 & -8 & 8 & -8 & 4 & 1 \\ 9 & -9 & 7 & -5 & 5 & -8 & -4 & -9 \\ 4 & 10 & 10 & -9 & 0 & 7 & -6 & -4 \\ -1 & -6 & 3 & 3 & 2 & 5 & -4 & -3 \\ 3 & -4 & -3 & -8 & -4 & 6 & -8 & 9 \\ -6 & -8 & 6 & 6 & -3 & -10 & 7 & -8 \end{bmatrix}$$

Matrix 3:

$$\begin{bmatrix} -10 & 5 & 5 & 0 & -7 & -3 & 1 & 3 & 0 & 7 & -2 & -2 & 3 & -6 & 7 \\ 1 & 2 & 0 & 7 & -6 & 0 & 9 & -9 & -2 & 3 & 5 & -3 & 9 & 1 & 2 \\ 0 & -9 & -10 & 8 & -3 & 7 & -7 & 3 & -2 & 5 & 5 & 2 & 3 & -8 & 8 \\ 5 & -10 & -1 & 3 & -5 & 10 & 1 & -6 & -8 & -9 & 2 & -2 & 0 & 1 & 7 \\ -4 & -7 & 1 & -8 & -9 & 8 & 8 & -2 & -10 & 8 & -9 & 0 & 1 & 10 & 8 \\ -3 & 4 & -4 & -9 & 10 & -8 & 9 & -7 & 6 & 3 & 2 & -9 & 9 & 0 & -5 \\ 7 & -2 & -3 & 3 & -5 & 5 & -10 & -5 & -7 & 2 & 3 & -3 & 9 & -6 & 4 \\ 10 & -8 & -7 & -2 & 10 & -4 & 9 & -9 & -2 & 9 & -2 & 2 & -4 & -8 & 4 \\ -10 & -9 & -3 & 9 & 3 & 2 & 3 & -10 & 5 & 3 & 9 & -1 & 6 & 3 & -9 \\ 6 & 5 & 7 & -3 & 10 & -8 & -6 & -9 & 3 & -6 & -3 & -10 & -6 & 4 & -6 \\ 2 & -10 & 10 & 7 & -3 & 2 & 4 & -2 & -5 & 7 & 9 & -3 & -4 & -3 & 7 \\ -2 & -2 & -5 & 5 & 9 & 3 & 0 & 1 & -9 & -8 & 10 & -4 & -7 & -6 & -1 \\ 8 & -4 & 10 & -8 & -3 & 4 & -6 & 1 & -2 & 10 & 3 & -1 & -10 & -7 & -1 \\ 4 & -1 & 1 & -4 & -3 & 8 & 4 & 8 & -3 & -2 & 3 & -10 & 8 & 10 & -2 \\ -3 & 9 & 6 & 6 & 1 & -10 & 5 & -1 & 9 & -5 & -8 & 1 & -7 & -6 & -5 \end{bmatrix}$$

Matrix 4:

$$\begin{bmatrix} -8 & -10 & -8 & 10 & -3 & 0 & 3 & 3 & 2 & -10 & -8 \\ -6 & 9 & -6 & -6 & -8 & 7 & -5 & -6 & -2 & -3 & 1 \\ -7 & -4 & -4 & -8 & -4 & 5 & -5 & -8 & 3 & -9 & -5 \\ -10 & -6 & -2 & 1 & -10 & 1 & -6 & -9 & -9 & -10 & 1 \\ 0 & -4 & -3 & -6 & -6 & 7 & -1 & -2 & 10 & -7 & 3 \\ 4 & -9 & 3 & 6 & -3 & -10 & 1 & -2 & -9 & -5 & -9 \\ 10 & 9 & -10 & -5 & -3 & 7 & -7 & -1 & 8 & 2 & -5 \\ 5 & -3 & 6 & -5 & -6 & 8 & 9 & 3 & -1 & 10 & -3 \\ 3 & -7 & -1 & 10 & 3 & 3 & 2 & -8 & -9 & 6 & 0 \\ -10 & 9 & 10 & 4 & 0 & 10 & -10 & -2 & 6 & 1 & -8 \\ 7 & -4 & 3 & -2 & -7 & -4 & -4 & -8 & -1 & -1 & 3 \end{bmatrix}$$

Matrix 5:

$$\begin{bmatrix} -5 & -7 & 1 & -4 & 8 \\ 9 & -7 & -9 & 0 & 7 \\ -4 & 0 & -9 & 6 & 7 \\ 0 & -5 & -2 & 0 & -10 \\ -6 & -8 & 3 & 1 & -6 \end{bmatrix}$$

Matrix * is a square matrix of size 30, it is not practical to display it here.

For each of the generated matrices, we have observed the following information:

Matrix no.	Size	Gauss-Jordan Reduction	LU Factorization	Iterative Method / No. of iterations
1	20	0.104000 sec	0.017000 sec	0.029000 sec / 17
2	8	0.008000 sec	0.003000 sec	0.006000 sec / 20
3	15	0.034000 sec	0.007000 sec	0.015000 sec / 20
4	10	0.017000 sec	0.005000 sec	0.011000 sec / 14
5	5	0.003000 sec	0.003000 sec	0.003000 sec / 12
*	30	0.250000 sec	0.031000 sec	0.071000 sec / 20

The performance of the methods are almost the same for small matrices, but they differ for large matrices. We expected that the iterative method will perform fastest since it has quadratic convergence. However, the LU Factorization method outperforms the other methods for large matrices. Therefore we should use this method for matrix inversion. Another aspect of the iterative method being slower than LU factorization is that the initial guess $\mathbf{X}_0$ is not good enough. In fact, a journal from American Mathematical Society [<http://www.ams.org/journals/mcom/1966-20-095/S0025-5718-66-99922-4/S0025-5718-66-99922-4.pdf>] suggests that the initial guess for the iterative method should be

$$\mathbf{X}_0 = \alpha \mathbf{A}^T$$

where

$$0 < \alpha < \frac{2}{\lambda_1(\mathbf{A}\mathbf{A}^T)}$$

and $\lambda_1(\mathbf{A}\mathbf{A}^T)$ is the largest eigenvalue of $\mathbf{A}\mathbf{A}^T$.

The `myIter2()` function in the source code implements the iterative method with $\mathbf{X}_0$ given above, with $\alpha = \frac{1}{\lambda_1(\mathbf{A}\mathbf{A}^T)}$. This requires less number of iterations for convergence than Heath's initial guess. However, the LU factorization method still performs better.

2 LLSE vs NLLSE

2.1 Method of orthogonal transformations

Given set of points (x_i, y_i) , the goal is to find $g(x)$, the best-fit function that will represent the data points such that if h_i is the actual y-value of the data point at $x = x_i$, then the linear sum of squares of the error $\xi_i = g(x_i) - h_i$ is minimized.

The method of orthogonal transformations will be used in solving this problem. Since the orbit is elliptical, we will derive an equation for the planet's orbit by solving the parameter vector of the general form equation of an ellipse. We will express the problem in matrix form $Ap = b$ where p is the parameter vector.

The general form of an ellipse is :

$$Bx^2 + Cxy + Dy^2 + Ex + Fy + G = 0$$

Dividing the whole equation by G ,

$$\begin{aligned}\bar{B}x^2 + \bar{C}xy + \bar{D}y^2 + \bar{E}x + \bar{F}y + 1 &= 0 \\ \bar{B}x^2 + \bar{C}xy + \bar{D}y^2 + \bar{E}x + \bar{F}y &= -1\end{aligned}$$

The equation above is already linear. It means that we can write the equation in matrix form $Ap = b$ where p is the parameter vector.

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 \\ x_2^2 & x_2y_2 & y_2^2 & x_2 & y_2 \\ x_3^2 & x_3y_3 & y_3^2 & x_3 & y_3 \\ x_4^2 & x_4y_4 & y_4^2 & x_4 & y_4 \\ x_5^2 & x_5y_5 & y_5^2 & x_5 & y_5 \\ x_6^2 & x_6y_6 & y_6^2 & x_6 & y_6 \\ x_7^2 & x_7y_7 & y_7^2 & x_7 & y_7 \\ x_8^2 & x_8y_8 & y_8^2 & x_8 & y_8 \\ x_9^2 & x_9y_9 & y_9^2 & x_9 & y_9 \\ x_{10}^2 & x_{10}y_{10} & y_{10}^2 & x_{10} & y_{10} \end{bmatrix} \times \begin{bmatrix} \bar{B} \\ \bar{C} \\ \bar{D} \\ \bar{E} \\ \bar{F} \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix}$$

Plugging in the values from the given points :

$$\begin{bmatrix} 1.02^2 & 1.02 \times 0.39 & 0.39^2 & 1.02 & 0.39 \\ 0.95^2 & 0.85 \times 0.32 & 0.32^2 & 0.95 & 0.32 \\ 0.87^2 & 0.87 \times 0.27 & 0.27^2 & 0.87 & 0.27 \\ 0.77^2 & 0.77 \times 0.22 & 0.22^2 & 0.77 & 0.22 \\ 0.67^2 & 0.67 \times 0.18 & 0.18^2 & 0.67 & 0.18 \\ 0.56^2 & 0.56 \times 0.15 & 0.15^2 & 0.56 & 0.15 \\ 0.44^2 & 0.44 \times 0.13 & 0.13^2 & 0.44 & 0.13 \\ 0.30^2 & 0.30 \times 0.12 & 0.12^2 & 0.30 & 0.12 \\ 0.16^2 & 0.16 \times 0.13 & 0.13^2 & 0.16 & 0.13 \\ 0.01^2 & 0.01 \times 0.15 & 0.15^2 & 0.01 & 0.15 \end{bmatrix} \times \begin{bmatrix} \bar{B} \\ \bar{C} \\ \bar{D} \\ \bar{E} \\ \bar{F} \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix}$$

To solve this linear system $Ap = b$, we get the QR decomposition of the matrix A . Note that $Q^T Q = I$ and $\|Q_y\| = \|y\|$ and R is an upper triangular matrix.

$$\begin{aligned}A &= QR \\ Ap &= QRp = b\end{aligned}$$

The implementation of the QR factorization is attached to the scilab script **Problem2-1.sci**. The QR factorization of matrix A produced the ff: matrices :

$$Q = \begin{bmatrix} -0.5846 & 0.6796 & 0.0047 & -0.0244 & -0.3783 & -0.0851 & -0.1341 & -0.086 & -0.0284 & -0.1402 \\ -0.5071 & 0.0203 & 0.0671 & 0.1963 & 0.3937 & 0.1725 & 0.2595 & 0.2112 & 0.2195 & 0.5954 \\ -0.4253 & -0.2609 & 0.0225 & 0.1724 & 0.3417 & 0.1419 & 0.1832 & 0.0747 & -0.2271 & -0.7023 \\ -0.3331 & -0.4068 & -0.0403 & 0.0526 & 0.0285 & -0.2914 & -0.4147 & -0.5013 & -0.3658 & 0.2763 \\ -0.2522 & -0.4141 & -0.0751 & -0.1101 & -0.2812 & -0.4387 & -0.1183 & 0.3518 & 0.5676 & -0.1253 \\ -0.1762 & -0.2928 & -0.0542 & -0.3224 & -0.3679 & 0.7804 & -0.1551 & -0.0401 & 0.0625 & 0.0483 \\ -0.1088 & -0.1067 & -0.0044 & -0.5127 & -0.1670 & -0.2128 & 0.7432 & -0.2463 & 0.0625 & 0.0483 \\ -0.0506 & 0.0808 & -0.0065 & -0.5885 & 0.2599 & -0.1009 & -0.2789 & 0.5580 & -0.4134 & 0.0823 \\ -0.0144 & 0.1693 & -0.2509 & -0.4218 & 0.5203 & 0.0286 & -0.1751 & -0.4240 & 0.4748 & -0.1512 \\ -0.0001 & 0.0203 & -0.9601 & 0.1588 & -0.0617 & 0.0117 & 0.1014 & 0.1200 & -0.1433 & 0.0586 \end{bmatrix}$$

$$R = \begin{bmatrix} -1.7797356 & -0.5966001 & -0.2029168 & -2.037538 & -0.6722965 \\ 0. & 0.0721568 & 0.0494166 & -0.264499 & 0.0140506 \\ 0. & 0. & -0.0223774 & -0.0771265 & -0.1790827 \\ 0. & 0. & 0. & -0.3701765 & -0.1250072 \\ 0. & 0. & 0. & 0. & 0.0390208 \\ 0. & 0. & 0. & 0. & 0 \\ 0. & 0. & 0. & 0. & 0 \\ 0. & 0. & 0. & 0. & 0 \\ 0. & 0. & 0. & 0. & 0 \\ 0. & 0. & 0. & 0. & 0 \end{bmatrix}$$

Pre-multiplying both sides by Q^T

$$Q^T Q R p = Q^T b$$

Since $Q^T Q = I$,

$$R p = Q^T b$$

$$p = \text{BackSubstitution}(R, p)$$

Since R is upper triangular, back substitution is used to obtain the values for the parameter vector. The implementation of the back substitution is also attached to the scilab script **Problem2-1.sci**. After back substitution, the computed p values are:

$$\begin{bmatrix} \bar{B} \\ \bar{C} \\ \bar{D} \\ \bar{E} \\ \bar{F} \end{bmatrix} = \begin{bmatrix} 2.2537948 \\ 0.0063247 \\ 5.5221834 \\ -1.2898102 \\ -7.3773544 \end{bmatrix}$$

The equation of the planet's orbit is:

$$g(x, y) = 2.2537948x^2 + 0.0063247xy + 5.5221834y^2 - 1.2898102x - 7.3773544y = -1$$

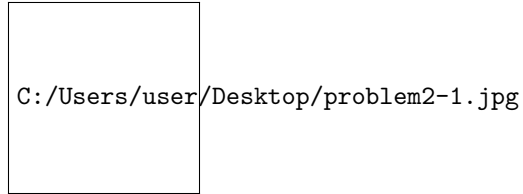


Figure 1: Plot of $2.2537948x^2 + 0.0063247xy + 5.5221834y^2 - 1.2898102x - 7.3773544y = -1$

To check if the equation above is a good representation of the planet's orbit, we plug-in the given data points to the equation and check how far it is from -1.

point	evaluation
(1.02,0.39)	- 1.0054864
(0.95,0.32)	- 0.984629
(0.87,0.27)	- 1.0040704
(0.77,0.22)	- 1.0115518
(0.67,0.18)	- 1.0006866
(0.56,0.15)	- 0.9973264
(0.44,0.13)	- 0.9965512
(0.30,0.12)	- 0.9896369
(0.16,0.13)	- 1.0142721
(0.01,0.15)	- 0.9950173

Note that the computed values for $g(x,y)$ are all very near to -1. It means that the errors of the function that we obtained through orthogonal transformation are very small.

3 Interpolation

3.1 Interpolating Specific heat

The attached file `ProblemThree.sce` contains the source code needed to perform interpolation on the given data. We should note here that the specific heat (c_p) J/gK of a material is dependent on temperature (T) in K .

$$f: T \rightarrow c_p$$
$$c_p = f(T)$$

For a given set of data points (x_i, y_i) , $i = 1, \dots, n$, the Newton interpolating function is given by

$$f(x) = a_1 + a_2(x - x_1) + a_3(x - x_1)(x - x_2) + \dots + a_n(x - x_1)(x - x_2)\dots(x - x_{n-1}).$$

To interpolate a point, we first solve all the a_i 's using a triangular linear system or the forward divided differences. However, using divided differences is easier to compute so we used it for implementation. From our reference book, it has form

$$a_k = f[t_1, t_2, \dots, t_k] = \frac{f[t_2, t_3, \dots, t_k] - f[t_1, t_2, \dots, t_{k-1}]}{t_k - t_1}$$
$$f[t_k] = y_k, k = 1, \dots, n$$

The divided differences are stored in a matrix to encourage dynamic programming. We used the forward (or bottom-up) approach of dynamic programming to prevent stack overflow in cases when the recursive calls are very deep (i.e, we have large number of data points).

After solving for a_i 's we compute $f(x)$ using Horner's Rule:

$$f(x) = f_i = a_{n-i} + (x - x_{n-i})f_{n-i}, i = 0, \dots, n-1$$
$$f_0 = a_n$$

Again, we used the bottom-up approach. Step by step calculations are printed in the Scilab console prior to the execution of the script.

Moving on to the Cubic Spline interpolation, the interpolating function from the book [*Numerical Methods for Scientists and Engineers 2e, Hoffman*] is given by

$$f_i(x) = \frac{f''_i}{6(x_{i+1} - x_i)}(x_{i+1} - x)^3 + \frac{f''_{i+1}}{6(x_{i+1} - x_i)}(x - x_i)^3 + \left[\frac{f_i}{x_{i+1} - x_i} - \frac{f''_i(x_{i+1} - x_i)}{6} \right] (x_{i+1} - x)$$
$$+ \left[\frac{f_{i+1}}{x_{i+1} - x_i} - \frac{f''_{i+1}(x_{i+1} - x_i)}{6} \right] (x - x_i)$$

Contrary to the formula used in class, we used this to fix issues regarding with the indexes in the subscripts. However, they are essentially the same. The second derivatives f''_i , $i = 2, 3, \dots, n-1$ can be solved using the equation

$$(x_i - x_{i+1})f''_{i-1} + 2(x_{i+1} - x_{i-1})f''_i + (x_{i+1} - x_i)f''_{i+1} = 6 \left[\frac{f_{i+1} - f_i}{x_{i+1} - x_i} - \frac{f_i - f_{i-1}}{x_i - x_{i-1}} \right]$$

with $f''_1 = f''_n = 0$. Solving for the second derivatives employs the use of tridiagonal matrices. We present here the interpolated values at intervals of 10 K graphically versus the true values presented in

http://www.kayelaby.npl.co.uk/general_physics/2_3/2_3_6.html.

C:/Users/user/Desktop/Interpolation.jpg

3.2 Errors in approximation

We use the two interpolating functions to predict the specific heat of aluminum when temperature is 273 K. We compute the the relative and absolute error using the following formula:

$$E_{rel} = \frac{|\hat{x} - x|}{x}$$
$$E_{abs} = |\hat{x} - x|$$

where $\hat{x}$ and x denotes the approximated and true values, respectively. The true value for $T = 273 K$ is $c_p = 0.880 J/gK$. For Newton, $c_p = 1.0131019 J/gK$, $E_{rel} = 17.1612\%$ and $E_{abs} = 15.1019\%$. While for Cubic Spline, $c_p = 1.027062 J/gK$, $E_{rel} = 16.7115\%$ and $E_{abs} = 14.7062\%$. It seems that the errors are bit large.

3.3 Inverse interpolation

Instead of finding the specific heat given the temperature, we do the reverse. Given the specific heat c_p , we should find the temperature T such that $f(T) = c_p$. This can be posed as a root-finding problem for the equation $f(T) - c_p = 0$.

We use the Regula-Falsi method for finding T since its convergence is super-linear. We chose the interval $T \in [150, 300]$ since $f(150) = 0.9666216$ and $f(300) = 1.04$ that satisfies the Intermediate Value Theorem. Using Regula Falsi method with $tol = 1 \times 10^{-5}$, $T = 211.952312 K$ for Newton, and $T = 216.975774 K$ for Cubic Spline.

4 Root-finding methods

5 Roots

5.1 Newton's Method

The Newton's method is a method for finding the zeroes of a real-valued function. In one variable, it is given by the iterative formula :

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

An initial guess of the root of the given function $f(x)$ is needed. For the iterative formula to converge, the initial guess should be near the actual root because the Newton's method inherits the properties of the Fixed-point method.

The efficiency equation of the given engine for monoatomic gas can be rewritten to a form that can easily be solved using the Newton's method. First step is to express the equation as a function of a single variable. Since the problem is looking for values of T_2/T_1 that will make $\eta = 0.3$, we can let $x = T_2/T_1$. Rewriting the given efficiency equation :

$$\eta = \frac{\ln(x) - (1 - \frac{1}{x})}{\ln(x) + \frac{(1 - \frac{1}{x})}{(\frac{5}{3} - 1)}}$$

Since the problem is to look for the value of x that will make η equals to 0.3, we can let $\eta = 0.3$.

$$0.3 = \frac{\ln(x) - (1 - \frac{1}{x})}{\ln(x) + \frac{(1 - \frac{1}{x})}{(\frac{5}{3} - 1)}}$$

By transposing 0.3 to the other side of the equation, the original problem will now be reduced to a single variable root finding problem. We let this function be $f(x)$.

$$0 = \frac{\ln(x) - (1 - \frac{1}{x})}{\ln(x) + \frac{(1 - \frac{1}{x})}{(\frac{5}{3} - 1)}} - 0.3 = \eta - 0.3 = f(x)$$

The original problem can be solved by solving this equation.

$$f(x) = 0 \text{ where } f(x) = \eta - 0.3 \text{ and } \eta = \frac{\ln(x) - (1 - \frac{1}{x})}{\ln(x) + \frac{(1 - \frac{1}{x})}{(\frac{5}{3} - 1)}}$$

Simplifying the equation,

$$f(x) = \frac{\ln(x) + \frac{1}{x} - 1 - \frac{3}{10}(\ln(x) - \frac{3}{2x} + \frac{3}{2})}{\ln(x) - \frac{3}{2x} + \frac{3}{2}} = 0$$

$$f(x) = \frac{\frac{29}{20x} + \frac{7\ln(x)}{10} - \frac{29}{20}}{\ln(x) - \frac{3}{2x} + \frac{3}{2}} = 0$$

In the given iterative formula for the Newton's method, the 1st derivative of the function is also needed. Solving for the 1st derivative of the function using quotient rule,

$$f'(x) = \frac{(\ln(x) - \frac{3}{2x} + \frac{3}{2})(\frac{-29}{20x^2} + \frac{7}{10x}) - (\frac{29}{20x} + \frac{7\ln(x)}{10} - \frac{29}{20})(\frac{1}{x} + \frac{3}{2x^2})}{(\ln(x) - \frac{3}{2x} + \frac{3}{2})^2}$$

Simplifying the equation above,

$$f'(x) = \frac{-\frac{5}{2x} - \frac{5\ln(x)}{2x^2} + \frac{5}{2x}}{(\ln(x))^2 + \frac{9}{4x^2} - \frac{12\ln(x)+18}{4x} + 3\ln(x) + \frac{9}{4}}$$

Now that we have defined $f(x)$ and its first derivative, we need an initial value of x to start the iteration. Since both equations have x in the denominator and since both equations has $\ln(x)$ functions, we cannot start with $x=0$ and $x=1$. Also, since $\ln(x)$ is undefined for $x \leq 0$, we must choose a positive number to start the iteration.

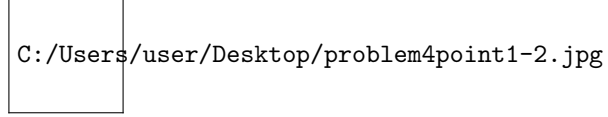


Figure 2: Graph of $f(x)$

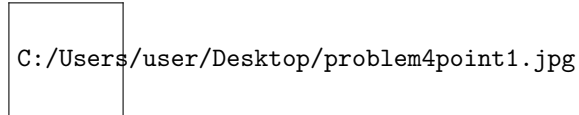


Figure 3: Graph of $f(x)$

In the attached scilab script **Problem 4-1.sci**, x_{old} is set to 6. As you can see in the graph of the function, the root is near 6. So, 6 seems to be a good choice for the initial value of x .

Two functions are defined in the script, the $f(x)$ and $f'(x)$. The Newton's method will use these 2 functions to evaluate the iterative formula. The stopping condition in the script for the iteration is if $|f(x_{old})| < 10^{-6}$

After 3 iterations, the Newton's method converges to the root.

Iteration number	x_{old}	$f(x_{old})$	x_{new}
1	6	0.0150894	5.3733616
2	5.3733616	-0.0010794	5.4123696
3	5.4123696	-0.0000049	5.4125482

After the 3rd iteration, $x_{old} = 5.4125482$ and $f(x_{old}) = 0.00000000113$ which already satisfied the stopping condition $|f(x_{old})| < 10^{-6}$.

5.2 Secant method

Secant method is a root-finding method that successively uses the zeroes of the secant lines to approximate the roots of a real-valued function. The iterative formula for the secant method is:

$$x_n = x_{n-1} - \frac{f(x_{n-1})(x_{n-1} - x_{n-2})}{f(x_{n-1}) - f(x_{n-2})}$$

As can be seen from the iterative formula, secant method requires 2 initial points x_0 and x_1 . Ideally, the 2 initial points should be close to the root.

In the given problem, the roots of the frequency equation $f(\beta) = \cosh(\beta)\cos(\beta) + 1 = 0$ is related to the natural frequencies of a uniform cantilever beam. The given frequency equation is a root finding problem that can be solved using the secant method.

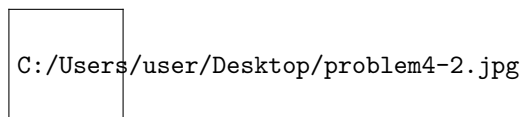


Figure 4: Graph of $f(\beta)$

By inspecting the graph of the frequency equation, it can be seen that there are four distinct roots (β_i). After solving for the roots, we can substitute the β_i 's to the equation $\beta_i^4 = (2\pi f_i)^2 \frac{mL^3}{EI}$ to obtain the corresponding f_i 's. To get the lowest 2 frequencies, we will just compare the values of the computed f_i 's

First step is to select 2 points that are close to the four roots. By inspection of the graph, the following points are selected:

Root	1st point	2nd point
leftmost root	-5	-5.5
2nd from the left	-2	-2.5
3rd from the left	1.5	1
rightmost root	4.5	4

In the attached scilab script **Problem4-2.sci**, The secant method is implemented as a function with the 2 initial points as its parameters. The β_i 's are computed by calling and passing the 2 initial points to the secantmethod function. After the execution, the script produces the ff. values of β .

Root	point
leftmost root	- 4.6940911
2nd from the left	- 1.8751041
3rd from the left	1.8751041
rightmost root	4.6940911

Now that the roots are computed, the corresponding f_i 's can be computed by substituting it to the given formula $\beta_i^4 = (2\pi f_i)^2 \frac{mL^3}{EI}$. The moment of inertia of the cross section I can be computed using the formula $I = \frac{wh^3}{12}$ where w and h are the width and height of the cross section respectively. Since the width of the cross section is 25 mm (0.025m) and the height is 2.5mm (0.0025m), then, $I = \frac{(0.025m)(0.0025m)^3}{12} = 3.255 \times 10^{-11}$. The volume of the steel beam with a rectangular cross section can be computed as $v = (length)(width)(height) = (0.9m)(0.025m)(0.0025m) = 0.0000563m^3$. The mass of the beam can be computed using the formula $m = (density)(volume) = (7850 \frac{kg}{m^3})(0.0000563m^3) = 0.441955kg$. Simplifying the equation and expressing in terms of f_i ,

$$\begin{aligned}\beta_i^4 &= (2\pi f_i)^2 \frac{mL^3}{EI} \\ f_i &= \sqrt[2]{\frac{\beta_i^4 EI}{4\pi^2 mL^3}} \\ f_i &= \sqrt[2]{\frac{\beta_i^4 EI}{4\pi^2 mL^3}} \\ f_i &= \sqrt[2]{\frac{\beta_i^4 (200)(3.225 \times 10^{-11})}{4\pi^2 (0.441955)(0.9)^3}} \\ f_i &= \sqrt[2]{5.123 \times 10^{-10} \beta_i^4}\end{aligned}$$

Substituting the values of the computed β_i 's to the equation above, the following values of f_i 's are obtained :

root position	Beta / Root	corresponding f_i
leftmost root	- 4.6940911	0.0017779
2nd from the left	- 1.8751041	0.0002837
3rd from the left	1.8751041	0.0002837
rightmost root	4.6940911	0.0017779

By inspection, the lowest two frequencies of the steel beam are **0.0002837** and **0.0017779**.